

Variables in Python

1. What is a Variable?

- A variable is like a container that stores data values.
- It acts as a **name** (identifier) that refers to a value stored in memory.
- In Python, you don't need to declare the type of a variable explicitly. The interpreter automatically assigns the data type based on the value assigned.

Example:

```
x = 10    # Integer
```

```
name = "Atharva" # String
```

```
pi = 3.14    # Float
```

```
is_valid = True # Boolean
```

2. Features of Variables in Python

1. **Dynamic Typing** – You don't need to declare the type before using it.
 2. `a = 5`
 3. `a = "Hello"` # same variable can hold different types
 4. **Case Sensitive** – `age`, `Age`, and `AGE` are different variables.
 5. **Memory Management** – Python handles memory allocation and deallocation automatically using its garbage collector.
 6. **Variable Reassignment** – You can reassign new values at any time.
-

3. Rules for Naming Variables

- Must start with a **letter** or an **underscore** (`_`).
- Cannot start with a number.
- Can contain letters, numbers, and underscores (`a_z`, `0_9`).
- Cannot use Python **keywords** (`if`, `else`, `while`, `for`, etc.).
- Variable names should be meaningful.

Valid examples:

```
name = "John"
```

```
age = 25
```

```
_price = 100
```

```
total_marks = 480
```

Invalid examples:

```
2name = "John" # cannot start with a number
```

```
my-name = "John" # hyphen not allowed
```

```
class = "Python" # cannot use keywords
```

4. Types of Variables

a) Global Variables

- Declared **outside functions**.
- Accessible inside and outside functions.

```
x = 50 # global variable
```

```
def show():
```

```
    print(x)
```

```
show()
```

```
print(x)
```

b) Local Variables

- Declared **inside functions**.
- Exist only while the function is running.

```
def show():
```

```
    y = 10 # local variable
```

```
    print(y)
```

```
show()
```

```
# print(y) # Error: y not defined outside the function
```

c) Global vs Local Variable Example

```
x = 100 # global
```

```
def func():  
    x = 50 # local (inside function)  
    print("Local x:", x)
```

```
func()  
print("Global x:", x)
```

5. Constants in Python

- Python does not have true constants.
- By convention, we use **UPPERCASE** names for values that should not be changed.

```
PI = 3.14159  
GRAVITY = 9.8
```

6. Multiple Assignment

Python allows assigning values to multiple variables in one line.

```
a, b, c = 10, 20, 30  
print(a, b, c)
```

Assign same value to multiple variables

```
x = y = z = 100  
print(x, y, z)
```

7. Deleting a Variable

Use del keyword to delete a variable.

```
a = 10  
print(a)  
del a  
# print(a) # Error: variable not defined
```

8. Variable Scope

- **LEGB Rule** – Local, Enclosing, Global, Built-in (priority order for variable lookup).

Example:

```
x = "global"
```

```
def outer():
```

```
    x = "enclosing"
```

```
    def inner():
```

```
        x = "local"
```

```
        print(x) # local will be printed
```

```
    inner()
```

```
    print(x)     # enclosing will be printed
```

```
outer()
```

```
print(x)         # global will be printed
```

9. Best Practices for Variables

- Use **meaningful names** (marks, price) instead of single letters.
 - Use **snake_case** (like total_score) for readability.
 - Keep constants in **UPPERCASE**.
 - Avoid overwriting built-in function names (list, dict, str, etc.).
-